

Wine Classification

Pracownia Analizy Danych, czerwiec 2026

Martyna Pieczka, Magdalena Chamera

Wstęp

Dane można pobrać ze strony <https://archive.ics.uci.edu/dataset/109/wine>, interesuje nas plik wine.data. Nazwy kolumn nadajemy zgodnie z opisem zawartym w wine.names.

```
wine_df <- read.csv("wine.data", header=FALSE)

colnames(wine_df) <- c(
  "Class",
  "Alcohol",
  "Malic_acid",
  "Ash",
  "Alcalinity_of_ash",
  "Magnesium",
  "Total_phenols",
  "Flavanoids",
  "Nonflavanoid_phenols",
  "Proanthocyanins",
  "Color_intensity",
  "Hue",
  "OD280_OD315",
  "Proline"
)
```

W celu poprawnego wyświetlania polskich znaków na wykresach skorzystano z biblioteki showtext:

```
library(showtext)
```

Warning: package 'showtext' was built under R version 4.4.3

Loading required package: sysfonts

Loading required package: showtextdb

```
showtext_auto()
```

Wstępna analiza danych - opis struktur danych

W trakcie wstępnej analizy danych postępowaliśmy następująco:

1. Sprawdzenie brakujących wartości.

```
colSums(is.na(wine_df))
```

Class	Alcohol	Malic_acid
0	0	0
Ash	Alcalinity_of_ash	Magnesium
0	0	0
Total_phenols	Flavanoids	Nonflavanoid_phenols
0	0	0
Proanthocyanins	Color_intensity	Hue
0	0	0
OD280_OD315	Proline	
0	0	

2. Sprawdzenie zrównoważenia klas.

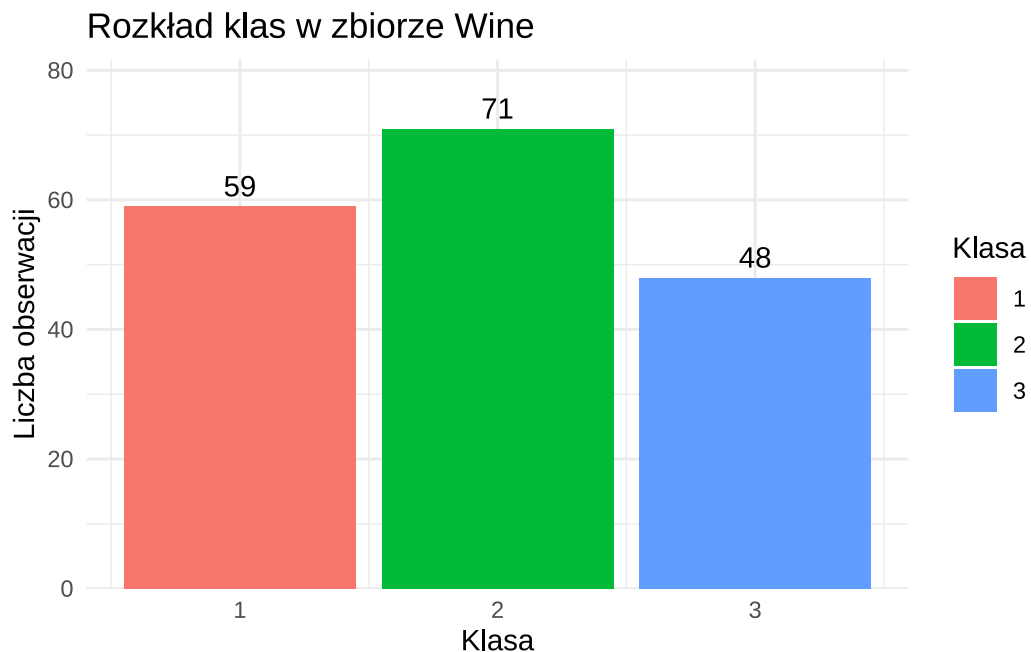
```
library(ggplot2)
```

Warning: package 'ggplot2' was built under R version 4.4.3

```
#tworzenie wykresu za pomocą ggplot2 - zliczanie
```

```
ggplot(wine_df, aes(x = Class, fill=as.factor(Class))) +  
  geom_bar() +  
  geom_text(stat = 'count', aes(label = after_stat(count)), vjust = -0.5) +  
  #wykres źle się skalował w pdf - dodanie 15% wolnej przestrzeni nad słupkiem  
  scale_y_continuous(expand = expansion(mult = c(0, 0.15))) +  
  theme_minimal() +  
  labs(title = "Rozkład klas w zbiorze Wine",
```

```
x = "Klasa",
y = "Liczba obserwacji",
fill = "Klasa")
```



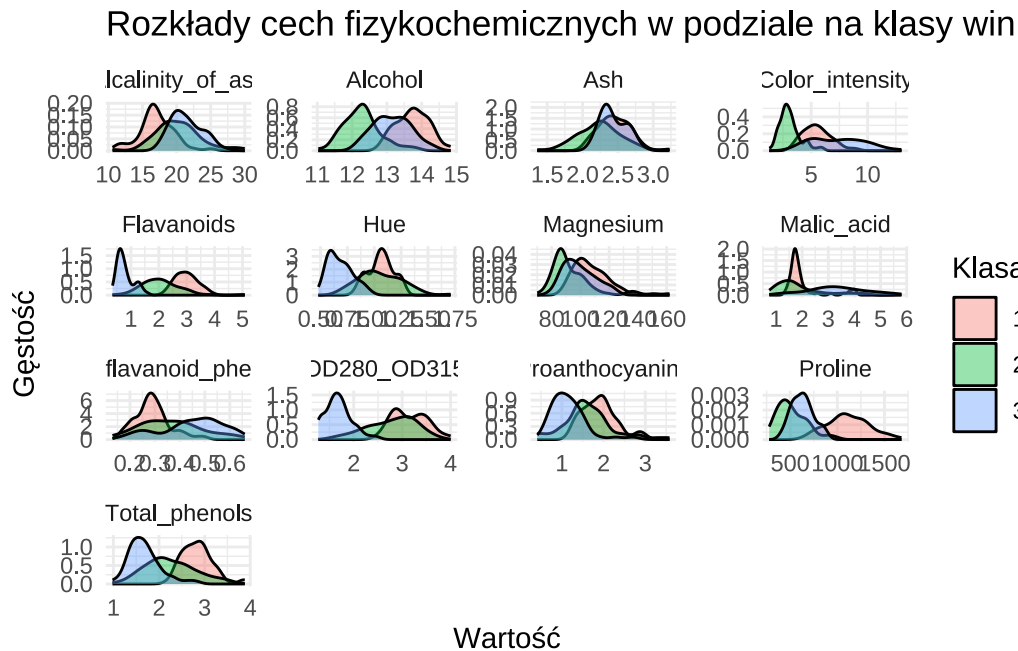
3. **Wykresy rozkładów zmiennych.** Można na ich podstawie wyznaczyć parametry, które izolują dane klasy:

- *Proline* ma znacznie wyższe wartości w **klasie 1** w porównaniu z pozostałymi klasami. *Malic_acid* ma bardzo niską wariancję dla klasy 1.
- *Alcohol*, *Color_intensity* są najniższe w **klasie 2**.
- W **klasie 3**, *Flavanoids*, *Hue*, *Total_Phenols* i *OD280_OD315* mają niskie wartości tych parametrów.

```
library(tidyr) #POTRZEBNY KOMENTARZ DO WYKRESU
```

```
#dane w formacie długim - potrzebne do wygenerowania wielu wykresów
wine_long <- pivot_longer(wine_df,
  cols = -Class,
  names_to = "Variable",
  values_to = "Value")
```

```
ggplot(wine_long, aes(x = Value, fill = as.factor(Class), group = as.factor(Class))) +
  geom_density(alpha = 0.4) +
  facet_wrap(~Variable, scales = "free") +
  theme_minimal() +
  labs(title = "Rozkłady cech fizykochemicznych w podziale na klasy win",
       x = "Wartość",
       y = "Gęstość",
       fill = "Klasa")
```



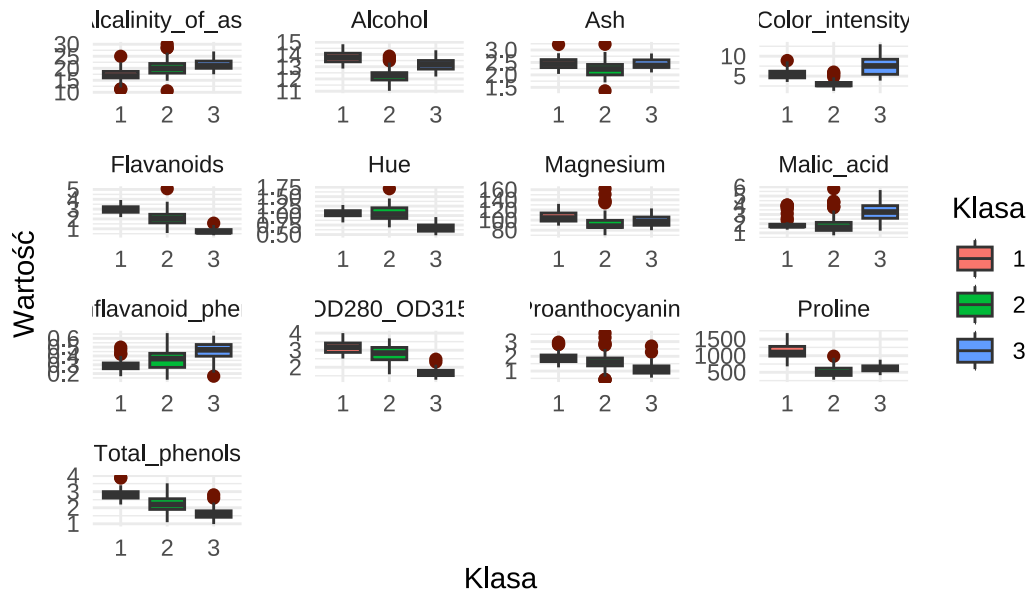
4. Wartości odstające.

- naruszone zostały założenia LDA o zbliżonej wariancji i braku wartości odstających. Macierze kowariancji mogą być zaburzone.
- klasy są od siebie fizycznie odseparowane (np. dla parametru *Flavanoids*), zatem model kNN oparty na odległościach, czy też drzewo decyzyjne oparte na regułach, znajdą czyste granice podziału dla tych zmiennych.

```
ggplot(wine_long, aes(x = as.factor(Class), y = Value, fill = as.factor(Class))) +
  geom_boxplot(outlier.colour = "#6E1300", outlier.shape = 16, outlier.size = 2) +
  facet_wrap(~Variable, scales = "free") +
  theme_minimal() +
  labs(title = "Analiza wartości odstających w podziale na klasy",
       x = "Klasa",
```

```
y = "Wartość",
fill = "Klasa")
```

Analiza wartości odstających w podziale na klasy



5. **Stworzenie wykresu korelacji.** Użyto metody sortowania hclust (hierarchichal clustering, które łączy zmienne w klastry na podstawie siły korelacji) wraz z dodawaniem trzech prostokątów. Wyodrębiono cztery grupy korelacji dodatnich, z czego najważniejsze są:

- klaster 1 (lewy górny róg):
 - zmienne *Hue*, *Proanthocyanins*, *OD280_OD315*, *Total_phenols*, *Flavanoids*
 - silna korelacja dodatnia, spowodowana zależnościami chemicznymi (flawonidy to związki z grupy polifenoli, wpływają na barwę i wynik testu OD280_OD315 na obecność białek i polifenoli).
- klaster 2:
 - zmienne *Magnesium*, *Color_intensity*, *Alcohol*, *Proline*
 - prolina jest aminokwasem znajdującym się w winogronach, którego stężenie wzrasta wraz z dojrzałością owoców. Przekłada się to na wyższy poziom cukru, w efekcie również alkoholu.

Klaster 3 i 4 są mniej istotne w analizie:

- klaster 3:
 - zmienne *Ash*, *Alcalinity_of_ash*

- powiązane parametry (*Ash* to sucha pozostałość po całkowitym spaleniu substancji organicznych w próbce. Składa się ze związków mineralnych. *Alcalinity_of_ash* mierzy zasadowość popiołu).
- klaster 4:
 - zmienne *Malic_acid*, *Nonflavanoid_phenols*
 - *Malic_acid*, czyli kwas jabłkowy, odpowiada za kwasowość wina, jego poziom spada w wyniku dojrzewania winogron na słońcu
 - **DOCZYTAĆ - ZOBACZYĆ STR XXII W KSIĄŻCE**

Ponadto, z klastrów można wyciągnąć następujące wnioski:

- Silne korelacje (zwłaszcza w klastrze 1) -> **warto zastosować PCA** przed tworzeniem modelu LDA.
- Drzewo decyzyjne najprawdopodobniej wybierze jako korzeń tylko jedną zmienną z pierwszego klastru - reszta nie wniesie żadnej nowej informacji.
- Ponieważ zmienne są skorelowane i opisują różne cechy w różnych jednostkach, należy wykonać **standaryzację danych** przed modelowaniem kNN.

```
library(corrplot)
```

corrplot 0.95 loaded

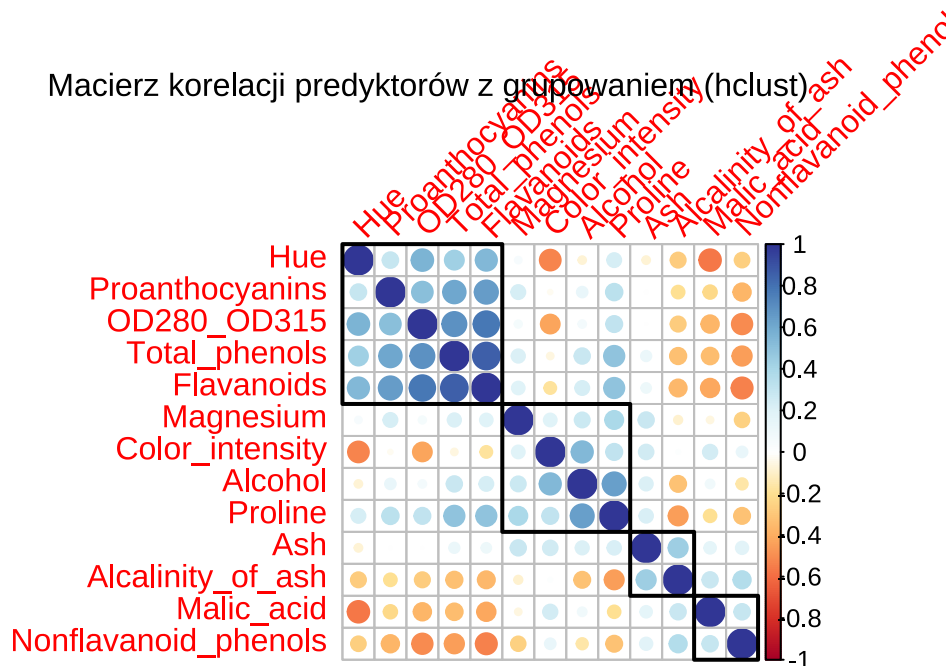
```
korelacje_wine <- cor(wine_df[, -1]) #wykluczamy kolumnę Class - zmienna dyskretna

corrplot(korelacje_wine,
  method = "circle",
  order = "hclust",
  addrect = 4,
  col = COL2("RdYlBu"),
  tl.srt = 45
)

#trzeba zaznaczyć tytuł wraz z poprzednim corrplot(). tutaj wygląda brzydko, w raporcie jest
#echo: false
# warning: false
# message: false
# fig-align: "center"
# fig-width: 8
# fig-height: 5
title(main = "Macierz korelacji predyktorów z grupowaniem (hclust)",
```

```
adj=0,
font.main = 1, # 2 oznacza pogrubienie (bold)
cex.main = 1) # rozmiar odpowiadający size = 16
```

Macierz korelacji predyktorów z grupowaniem (hclust)



Przygotowanie danych i tworzenie modeli

Metodologia

Będziemy stroić nasze modele. W związku z tym najpierw dokonamy podziału na zbiór uczący i testowy (w stosunku 70:30), a następnie ustawimy krosvalidację na zbiorze uczącym. Skorzystamy z 10-krotnej krosvalidacji.

Aby uniknąć data leakage, zastosujemy przekształcenia na 9 foldach treningowych i zaaplikujemy je na 1 fold walidacyjny w każdej iteracji. W ten sposób dziesiąty fold będzie standaryzowany używając średniej i odchylenia wyliczonych z danych treningowych - dopasowujemy zbiór testowy do skali, której nauczył się model.

Tworzymy i stroimy trzy osobne modele:

- kNN - będziemy tuningować k - liczbę sąsiadów. Wykonamy standaryzację i PCA
- LDA - wykonamy standaryzację oraz PCA.

- drzewo decyzyjne - będziemy tuningować *cp* - parametr złożoności. Nie będziemy transformować danych.

```
wine_df <- read.csv("wine.data", header=FALSE)

colnames(wine_df) <- c(
  "Class",
  "Alcohol",
  "Malic_acid",
  "Ash",
  "Alcalinity_of_ash",
  "Magnesium",
  "Total_phenols",
  "Flavanoids",
  "Nonflavanoid_phenols",
  "Proanthocyanins",
  "Color_intensity",
  "Hue",
  "OD280_OD315",
  "Proline"
)
wine_df$Class <- as.factor(wine_df$Class)

library(rsample)
```

Warning: package 'rsample' was built under R version 4.4.3

```
library(kknn)    #dla kNN
library(MASS)    #dla LDA
library(rpart)   #dla drzew decyzyjnych
library(ipred)   #dla drzew - do baggingu

set.seed(123)
#PODZIAŁ
#initial_split domyślnie zachowuje proporcje klas
wine_split <- initial_split(wine_df, prop = 0.7, strata = Class)
wine_train <- training(wine_split)
wine_test  <- testing(wine_split)

#KROSWALIDACJA
folds <- vfold_cv(wine_train, v = 10, strata = Class)
```



```

#siatki do tuningu hiperparametrów
k_grid <- c(1, 3, 5, 7, 9, 11, 13, 15)
#siatka kerneli i połączenie w jedną siatkę (kombinacje)
kernel_grid <- c("rectangular", "epanechnikov", "gaussian", "optimal")
knn_grid <- expand.grid(k = k_grid, kernel = kernel_grid)

cp_grid <- seq(0.001, 0.05, by = 0.005)
#siatka dla LDA (liczba uwzględnianych składowych głównych z PCA)
pca_grid <- 2:8

#macierze na wyniki tuningu (wiersze = foldy, kolumny = hiperparametry)
#liczba kolumn zależy teraz od liczby wszystkich kombinacji w knn_grid
acc_knn_mat <- matrix(0, nrow = 10, ncol = nrow(knn_grid))
acc_tree_mat <- matrix(0, nrow = 10, ncol = length(cp_grid))
acc_lda_mat <- matrix(0, nrow = 10, ncol = length(pca_grid))

#wektory do zapisu dokładności w każdym z 10 przebiegów pętli
acc_knn <- numeric(10)
acc_lda <- numeric(10)
acc_tree <- numeric(10)

#na wyniki predykcji
wyniki_pred <- data.frame()

#pętla krosvalidacji
for(i in 1:10) {

  #wyciągnięcie podziału dla danej iteracji
  fold_train <- analysis(folds$splits[[i]])
  fold_val <- assessment(folds$splits[[i]])

  #---DLA DRZEWA - dane surowe
  train_unscaled <- fold_train
  val_unscaled <- fold_val

  #---STANDARYZACJA I PCA
  X_train <- fold_train[, -1]
  Y_train <- fold_train$Class
  X_val <- fold_val[, -1]
  Y_val <- fold_val$Class

  #PCA z wbudowaną standaryzacją

```

```

pca_model <- prcomp(X_train, center = TRUE, scale. = TRUE)

#pobranie 5 składowych dla zbioru uczącego
X_train_pca <- pca_model$x[, 1:5]

#funkcja predict dla prcomp automatycznie standaryzuje X_val parametrami ze zbioru uczącego
X_val_pca <- predict(pca_model, newdata = X_val)[, 1:5]

#złączenie z powrotem w ramki danych
train_pca <- data.frame(Class = Y_train, X_train_pca)
val_pca <- data.frame(Class = Y_val, X_val_pca)

#-----TRENOWANIE I PREDYKCJA MODELI

#---Tuning kNN (pętla po siatce kombinacji knn_grid)
#iteracja przez wszystkie wiersze siatki knn_grid
for(j in 1:nrow(knn_grid)) {
  knn_model_temp <- kknn(Class ~ ., train = train_pca, test = val_pca,
                        k = knn_grid$k[j], kernel = as.character(knn_grid$kernel[j]))
  acc_knn_mat[i, j] <- mean(knn_model_temp$fitted.values == Y_val)
  #zapis predykcji dla pierwszej wartości z siatki (na potrzeby tabeli wyniki_pred i heatmap)
  if(j == 1) knn_pred <- knn_model_temp$fitted.values
}

#---Tuning LDA (pętla po siatce pca_grid)
for(j in seq_along(pca_grid)) {
  #pobranie dynamicznej liczby składowych z siatki
  num_comp <- pca_grid[j]
  train_pca_lda <- data.frame(Class = Y_train, pca_model$x[, 1:num_comp])
  val_pca_lda <- data.frame(Class = Y_val, predict(pca_model, newdata = X_val)[, 1:num_comp])

  lda_model_temp <- lda(Class ~ ., data = train_pca_lda)
  lda_pred_temp <- predict(lda_model_temp, newdata = val_pca_lda)$class
  acc_lda_mat[i, j] <- mean(lda_pred_temp == Y_val)

  #zapis predykcji dla pierwszej wartości z siatki (na potrzeby tabeli wyniki_pred i heatmap)
  if(j == 1) lda_pred <- lda_pred_temp
}

#---Tuning Drzewa decyzyjnego (pętla po siatce cp_grid)
for(j in seq_along(cp_grid)) {
  #przekazanie parametru cp przez rpart.control wewnątrz funkcji bagging

```

```

    tree_model_temp <- bagging(Class ~ ., data = train_unscaled, nbagg = 25, control = rpart
    tree_pred_temp <- predict(tree_model_temp, newdata = val_unscaled)
    acc_tree_mat[i, j] <- mean(tree_pred_temp == Y_val)
    #zapis predykcji dla pierwszej wartości z siatki (na potrzeby tabeli wyniki_pred i heatmap)
    if(j == 1) tree_pred <- tree_pred_temp
  }

  #zapis predykcji
  ID_wina <- rownames(val_pca)
  fold_results <- data.frame(
    ID = ID_wina,
    Rzeczywista_Klasa = Y_val,
    kNN = knn_pred,
    LDA = lda_pred,
    Tree = tree_pred
  )
  #PODSUMOWANIE WYNIKÓW

  wyniki_pred <- rbind(wyniki_pred, fold_results)
}

#wybór najlepszych hiperparametrów ---
mean_knn_acc <- colMeans(acc_knn_mat)
#wyciągnięcie optymalnego indeksu, a z niego k oraz kernel
best_knn_idx <- which.max(mean_knn_acc)
best_k <- knn_grid$k[best_knn_idx]
best_kernel <- as.character(knn_grid$kernel[best_knn_idx])

#wybór najlepszej liczby składowych dla LDA
mean_lda_acc <- colMeans(acc_lda_mat)
best_pca_idx <- which.max(mean_lda_acc)
best_pca <- pca_grid[best_pca_idx]

mean_tree_acc <- colMeans(acc_tree_mat)
best_cp_idx <- which.max(mean_tree_acc)
best_cp <- cp_grid[best_cp_idx]

cat("\n=== WYNIKI TUNINGU ===\n")

```

=== WYNIKI TUNINGU ===

```
#wydruk uwzględnia najlepszy kernel
cat("Najlepsze parametry kNN: k =", best_k, ", kernel =", best_kernel, "(Średnie Accuracy:",
```

Najlepsze parametry kNN: k = 9 , kernel = epanechnikov (Średnie Accuracy: 0.9833)

```
cat("Najlepsza 1. składowych (PCA) dla LDA:", best_pca, "(Średnie Accuracy:", round(mean_lda,
```

Najlepsza 1. składowych (PCA) dla LDA: 6 (Średnie Accuracy: 0.9756)

```
cat("Najlepsze cp dla Drzewa:", best_cp, "(Średnie Accuracy:", round(mean_tree_acc[best_cp_i,
```

Najlepsze cp dla Drzewa: 0.041 (Średnie Accuracy: 0.9099)

```
#nadpisanie wektorów wynikami z najlepszych modeli
acc_knn <- acc_knn_mat[, best_knn_idx]
acc_lda <- acc_lda_mat[, best_pca_idx]
acc_tree <- acc_tree_mat[, best_cp_idx]

wyniki_cv <- data.frame(
  Fold = 1:10,
  KNN_Acc = acc_knn,
  LDA_Acc = acc_lda,
  Tree_Acc = acc_tree
)

cat("Wyniki z poszczególnych foldów (dla optymalnych parametrów):\n")
```

Wyniki z poszczególnych foldów (dla optymalnych parametrów):

```
print(round(wyniki_cv, 4))
```

	Fold	KNN_Acc	LDA_Acc	Tree_Acc
1	1	1.0000	1.0000	0.9286
2	2	1.0000	0.9231	0.7692
3	3	1.0000	1.0000	1.0000
4	4	1.0000	0.9167	0.9167
5	5	1.0000	1.0000	0.9167
6	6	1.0000	1.0000	1.0000

7	7	1.0000	0.9167	0.9167
8	8	0.9167	1.0000	0.8333
9	9	0.9167	1.0000	1.0000
10	10	1.0000	1.0000	0.8182

```
cat("\nŚrednie Accuracy z krosvalidacji:\n")
```

Średnie Accuracy z krosvalidacji:

```
cat("kNN:   ", round(mean(acc_knn), 4), "\n")
```

kNN: 0.9833

```
cat("LDA:   ", round(mean(acc_lda), 4), "\n")
```

LDA: 0.9756

```
cat("Tree:  ", round(mean(acc_tree), 4), "\n")
```

Tree: 0.9099

Wykresy

Wykresy

```
library(ggplot2)
library(tidyr)
library(dplyr)
```

Attaching package: 'dplyr'

The following object is masked from 'package:MASS':

select

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
#przekształcenie danych do formatu długiego (wymóg ggplot2)
wyniki_long <- pivot_longer(wyniki_cv,
                             cols = c(KNN_Acc, LDA_Acc, Tree_Acc),
                             names_to = "Model",
                             values_to = "Accuracy")

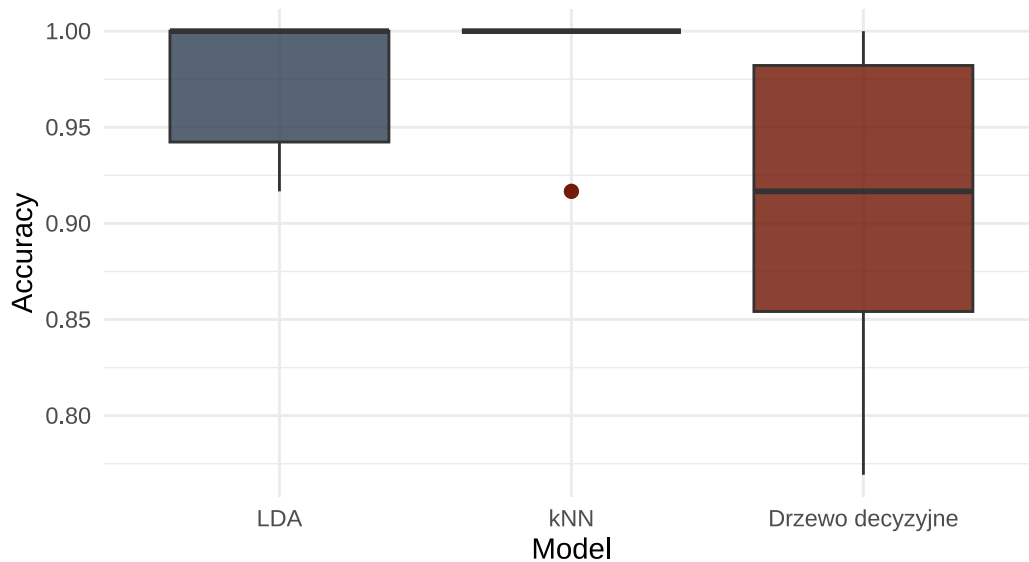
#poprawa nazw modeli dla lepszej czytelności na wykresie
wyniki_long$Model <- factor(wyniki_long$Model,
                             levels = c("LDA_Acc", "KNN_Acc", "Tree_Acc"),
                             labels = c("LDA", "kNN", "Drzewo decyzyjne"))

#WYKRES STABILNOŚCI NA FOLDACH
wykres_box <- ggplot(wyniki_long, aes(x = reorder(Model, -Accuracy, FUN = median), y = Accuracy)) +
  geom_boxplot(alpha = 0.8, outlier.colour = "#6E1300", outlier.size = 2) +
  scale_fill_manual(values = c("LDA" = "#2C3E50", "kNN" = "#18BC9C", "Drzewo decyzyjne" = "#E67E22")) +
  theme_minimal() +
  labs(title = "Rozkład dokładności w 10-krotnej krosvalidacji",
        subtitle = "Węższe pudełko oznacza większą stabilność modelu na różnych podpróbkach",
        x = "Model",
        y = "Accuracy") +
  theme(legend.position = "none")

print(wykres_box)
```

Rozkład dokładności w 10-krotnej krosvalidacji

Węższe pudełko oznacza większą stabilność modelu na różnych podpróbach

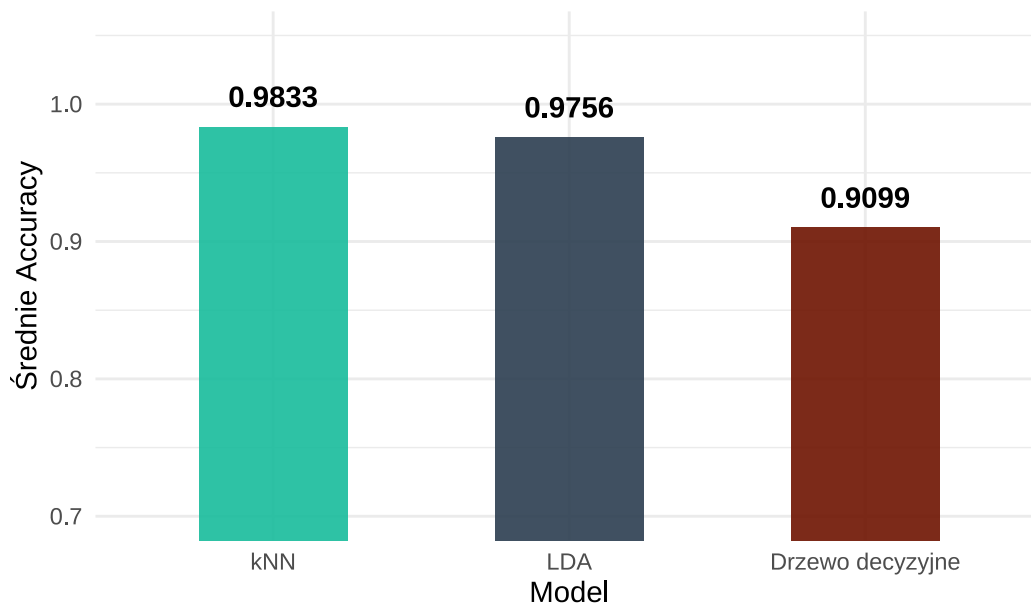


```
#ŚREDNIE ACCURACY
wyniki_mean <- wyniki_long %>%
  group_by(Model) %>%
  summarise(Mean_Accuracy = mean(Accuracy))

wykres_bar <- ggplot(wyniki_mean, aes(x = reorder(Model, -Mean_Accuracy), y = Mean_Accuracy,
  geom_col(width = 0.5, alpha = 0.9) +
  geom_text(aes(label = round(Mean_Accuracy, 4)), vjust = -1, fontface = "bold") +
  scale_fill_manual(values = c("LDA" = "#2C3E50", "kNN" = "#18BC9C", "Drzewo decyzyjne" = "#E67E22"),
  coord_cartesian(ylim = c(0.7, 1.05)) + #skracamy oś OY dla uwydatnienia różnicy między modelami
  theme_minimal() +
  labs(title = "Średnia dokładność (Accuracy) modeli",
        x = "Model",
        y = "Średnie Accuracy") +
  theme(legend.position = "none")

print(wykres_bar)
```

Średnia dokładność (Accuracy) modeli



```
#TRUDNE PRZYPADKI
trudne_przypadki <- wyniki_pred %>%
  filter(kNN != Rzeczywista_Klasa | LDA != Rzeczywista_Klasa | Tree != Rzeczywista_Klasa)

cat("Liczba win, z którymi modele miały problem:", nrow(trudne_przypadki), "\n")
```

Liczba win, z którymi modele miały problem: 21

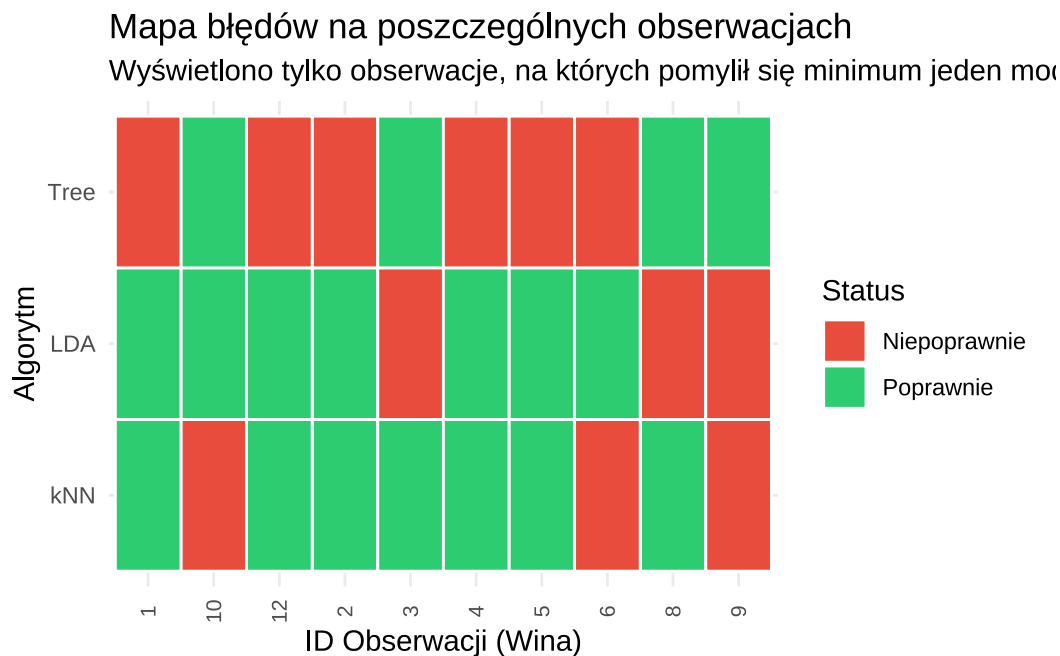
```
library(ggplot2)

#przekształcenie do wykresu
bledy_wykres <- trudne_przypadki %>%
  mutate(
    kNN = ifelse(kNN == Rzeczywista_Klasa, "Poprawnie", "Niepoprawnie"),
    LDA = ifelse(LDA == Rzeczywista_Klasa, "Poprawnie", "Niepoprawnie"),
    Tree = ifelse(Tree == Rzeczywista_Klasa, "Poprawnie", "Niepoprawnie")
  ) %>%
  dplyr::select(ID, kNN, LDA, Tree) %>% # <-- dodane dplyr::, inaczej scale() było z MASS i
  pivot_longer(-ID, names_to = "Model", values_to = "Status")

#generowanie heatmapy dla błędów
ggplot(bledy_wykres, aes(x = as.factor(ID), y = Model, fill = Status)) +
  geom_tile(color = "white", linewidth = 0.5) +
```



```
scale_fill_manual(values = c("Niepoprawnie" = "#E74C3C", "Poprawnie" = "#2ECC71")) +
theme_minimal() +
theme(axis.text.x = element_text(angle = 90, vjust = 0.5, size = 8)) +
labs(title = "Mapa błędów na poszczególnych obserwacjach",
      subtitle = "Wyświetlono tylko obserwacje, na których pomylił się minimum jeden model",
      x = "ID Obserwacji (Wina)",
      y = "Algorytm")
```



```
#wykres - przed i po tunignu
library(ggplot2)

# ---WYCIĄGNIĘCIE WYNIKÓW BAZOWYCH (PRZED TUNINGIEM)
#szukamy dokładnego indeksu w nowej siatce knn_grid, gdzie k=5 i kernel="rectangular"
idx_knn_bazowe <- which(knn_grid$k == 5 & knn_grid$kernel == "rectangular")
acc_knn_przed <- mean_knn_acc[idx_knn_bazowe]

#domyślne cp dla rpart to 0.01, więc znajdujemy w siatce wartość najbliższą tej liczbie
idx_tree_bazowe <- which.min(abs(cp_grid - 0.01))
acc_tree_przed <- mean_tree_acc[idx_tree_bazowe]

#wynik bazowy dla LDA (5 składowych PCA)
idx_lda_bazowe <- which(pca_grid == 5)
acc_lda_przed <- mean_lda_acc[idx_lda_bazowe]
```

```

#ZBUDOWANIE RAMKI DANYCH DO WYKRESU ---
df_porownanie <- data.frame(
  Model = rep(c("kNN", "Drzewo decyzyjne", "LDA"), each = 2),
  Etap = rep(c("Przed tuningiem", "Po tuningu"), times = 3),
  Accuracy = c(acc_knn_przed, mean_knn_acc[best_knn_idx], #wyniki optymalne
               acc_tree_przed, mean_tree_acc[best_cp_idx],
               acc_lda_przed, mean_lda_acc[best_pca_idx])
)

#zabezpieczenie przed błędem wyświetlania nazw modeli w odpowiedniej kolejności
df_porownanie$Model <- factor(df_porownanie$Model, levels = c("Drzewo decyzyjne", "LDA", "kNN"))

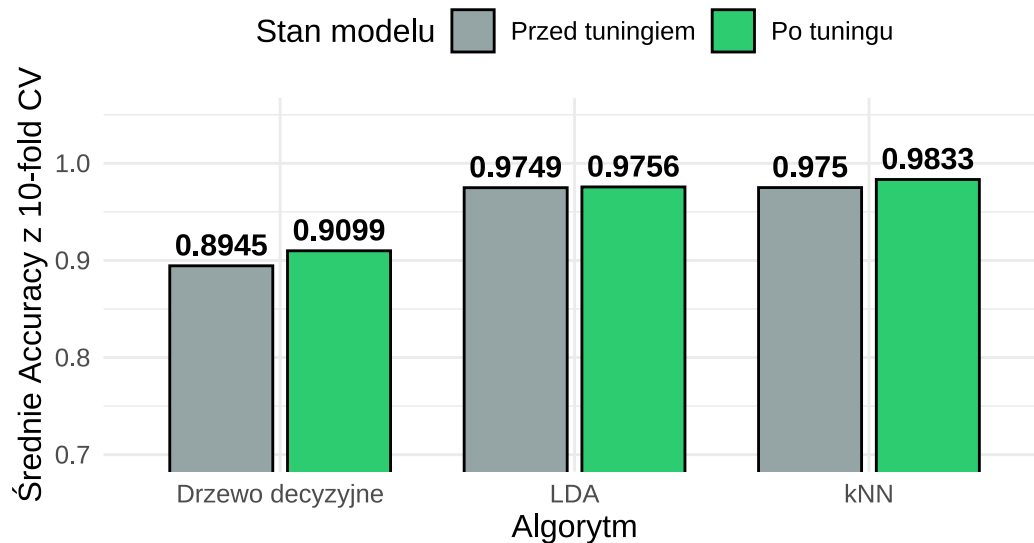
# DODANE: wymuszenie kolejności dla Etapu
df_porownanie$Etap <- factor(df_porownanie$Etap, levels = c("Przed tuningiem", "Po tuningu"))

#WYKRES
wykres_porownanie <- ggplot(df_porownanie, aes(x = Model, y = Accuracy, fill = Etap)) +
  geom_col(position = position_dodge(width = 0.8), width = 0.7, color = "black") +
  geom_text(aes(label = round(Accuracy, 4)),
            position = position_dodge(width = 0.8), vjust = -0.5, size = 4, fontface = "bold") +
  scale_fill_manual(values = c("Przed tuningiem" = "#95A5A6", "Po tuningu" = "#2ECC71")) +
  coord_cartesian(ylim = c(0.7, 1.05)) +
  theme_minimal() +
  labs(title = "Wpływ optymalizacji hiperparametrów na dokładność modeli",
       subtitle = paste("kNN (bazowe k=5 -> optymalne k=", best_k,
                        ") | Drzewo (cp~0.01 -> cp=", best_cp,
                        ") | LDA (PCA=5 -> PCA=", best_pca, ")"), sep=""),
       x = "Algorytm",
       y = "Średnie Accuracy z 10-fold CV",
       fill = "Stan modelu") +
  theme(legend.position = "top",
        text = element_text(size = 12))

print(wykres_porownanie)

```

Wpływ optymalizacji hiperparametrów na dokładność mc kNN (bazowe k=5 -> optymalne k=9) | Drzewo (cp~0.01 -> cp=0.04)



```
library(ggplot2)

#Wykres tuningu kNN (Accuracy vs k vs kernel)

#przygotowanie danych
# colMeans(acc_knn_mat) liczy średnią dla każdej z 32 kolumn (kombinacji k i kernel)
df_knn_tuning <- data.frame(
  k = knn_grid$k,
  Kernel = knn_grid$kernel,
  Accuracy = colMeans(acc_knn_mat)
)

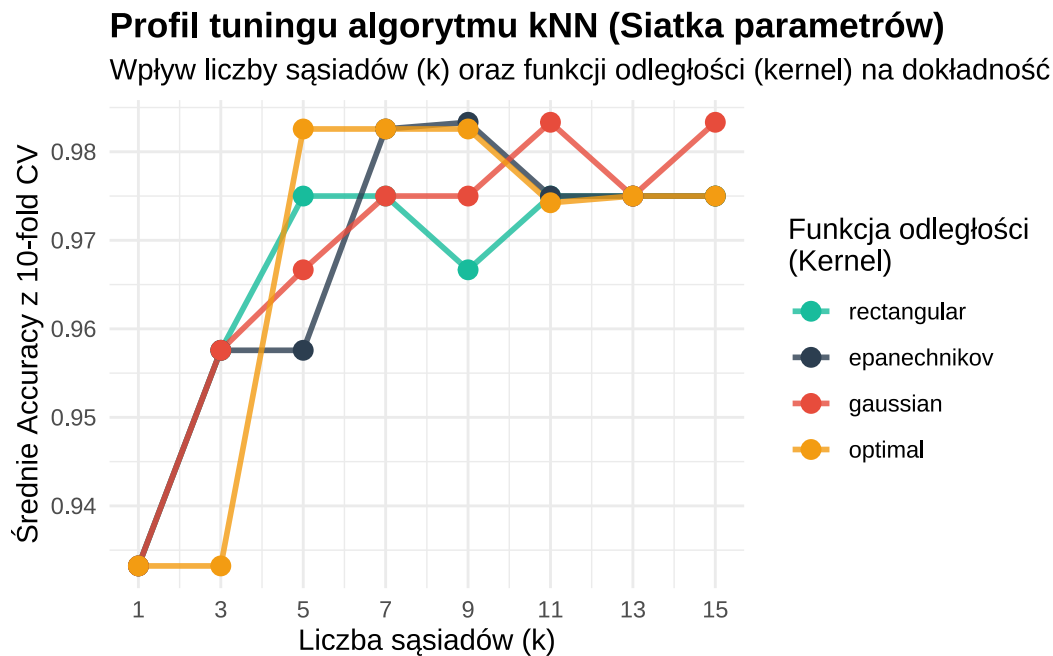
#wykres
wykres_knn_tuning <- ggplot(df_knn_tuning, aes(x = k, y = Accuracy, color = Kernel, group = Kernel)) +
  geom_line(linewidth = 1, alpha = 0.8) +
  geom_point(size = 3) +
  #wymuszenie wyświetlania tylko testowanych wartości 'k' na osi X
  scale_x_continuous(breaks = unique(df_knn_tuning$k)) +
  #ręczne przypisanie palety kolorów dla rozróżnienia kerneli
  scale_color_manual(values = c("rectangular" = "#18BC9C",
                                "epanechnikov" = "#2C3E50",
                                "gaussian" = "#E74C3C",
                                "optimal" = "#F39C12")) +
```

```

theme_minimal() +
labs(title = "Profil tuningu algorytmu kNN (Siatka parametrów)",
      subtitle = "Wpływ liczby sąsiadów (k) oraz funkcji odległości (kernel) na dokładność",
      x = "Liczba sąsiadów (k)",
      y = "Średnie Accuracy z 10-fold CV",
      color = "Funkcja odległości\\n(Kernel)") +
theme(legend.position = "right",
      plot.title = element_text(face = "bold"))

print(wykres_knn_tuning)

```



```

library(ggplot2)

#Wykres tuningu LDA (Accuracy vs Liczba składowych PCA)

#dane
df_lda_tuning <- data.frame(
  PCA_Components = pca_grid,
  Accuracy = colMeans(acc_lda_mat)
)

#wykres
wykres_lda_tuning <- ggplot(df_lda_tuning, aes(x = PCA_Components, y = Accuracy)) +

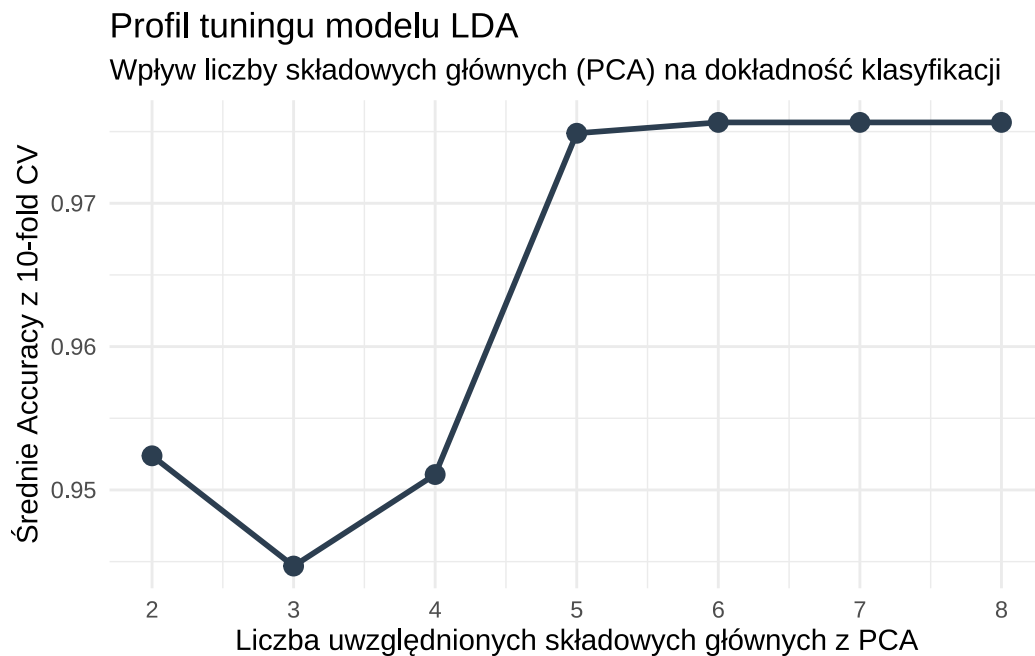
```

```

geom_line(color = "#2C3E50", linewidth = 1) +
geom_point(color = "#2C3E50", size = 3) +
#wymuszenie wyświetlania tylko całkowitych liczb na osi X
scale_x_continuous(breaks = pca_grid) +
theme_minimal() +
labs(title = "Profil tuningu modelu LDA",
      subtitle = "Wpływ liczby składowych głównych (PCA) na dokładność klasyfikacji",
      x = "Liczba uwzględnionych składowych głównych z PCA",
      y = "Średnie Accuracy z 10-fold CV")

print(wykres_lda_tuning)

```



```

#Wykres tuningu Drzewa (Accuracy vs cp)

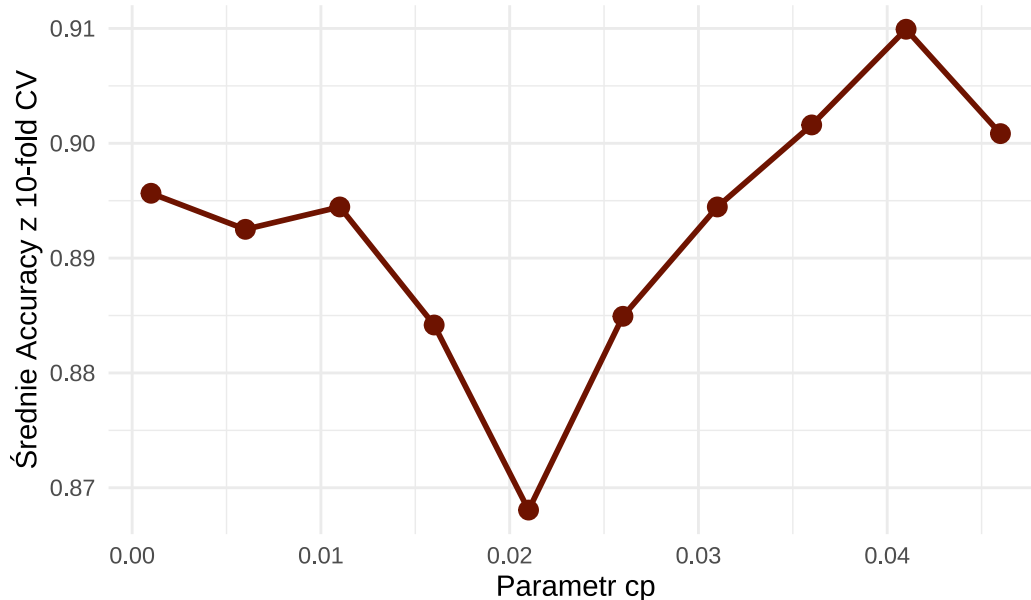
#dane
df_tree_tuning <- data.frame(cp = cp_grid, Accuracy = colMeans(acc_tree_mat))

#wykres
ggplot(df_tree_tuning, aes(x = cp, y = Accuracy)) +
  geom_line(color = "#6E1300", linewidth = 1) +
  geom_point(color = "#6E1300", size = 3) +
  theme_minimal() +
  labs(title = "Profil tuningu parametru złożoności drzewa (cp)",

```

```
x = "Parametr cp",
y = "Średnie Accuracy z 10-fold CV")
```

Profil tuningu parametru złożoności drzewa (cp)



```
#EWALUACJA MODELI NA ZBIORZE TESTOWYM (WINE_TEST)

#przygotowanie pełnych danych treningowych i testowych w przestrzeni PCA (dla kNN i LDA)
X_train_full <- wine_train[, -1]
Y_train_full <- wine_train$Class
X_test_full  <- wine_test[, -1]
Y_test_full  <- wine_test$Class

#budowa ostatecznego modelu PCA na całym zbiorze treningowym
pca_full <- prcomp(X_train_full, center = TRUE, scale. = TRUE)

#projekcja na optymalną liczbę składowych głównych best_pca
train_pca_full <- data.frame(Class = Y_train_full, pca_full$x[, 1:best_pca])
test_pca_full  <- data.frame(Class = Y_test_full, predict(pca_full, newdata = X_test_full)[,

#trenowanie finałowych modeli na całych dostępnych danych treningowych

#Finałowy model kNN (uwzględnia wybrane best_k oraz best_kernel)
final_knn <- kknn(Class ~ ., train = train_pca_full, test = test_pca_full, k = best_k, kerne
```

```

pred_test_knn <- final_knn$fitted.values

#Finałowy model LDA (działa automatycznie na optymalnej liczby składowych z train_pca_full)
final_lda <- lda(Class ~ ., data = train_pca_full)
pred_test_lda <- predict(final_lda, newdata = test_pca_full)$class

#Finałowy model Drzewa z Baggingiem (z optymalnym cp na surowych danych)
final_tree <- bagging(Class ~ ., data = wine_train, nbagg = 25, control = rpart.control(cp =
pred_test_tree <- predict(final_tree, newdata = wine_test)

#Wyświetlenie macierzy pomyłek i wyników Accuracy - graficzne
#tworzenie bazowych macierzy pomyłek
cm_knn <- table(Rzeczywista = Y_test_full, Przewidywana = pred_test_knn)
cm_lda <- table(Rzeczywista = Y_test_full, Przewidywana = pred_test_lda)
cm_tree <- table(Rzeczywista = Y_test_full, Przewidywana = pred_test_tree)

#GRAFICZNE MACIERZE POMYŁEK na jednym wykresie
library(patchwork)

```

Attaching package: 'patchwork'

The following object is masked from 'package:MASS':

area

```

library(ggplot2)

rysuj_macierz <- function(cm, tytul, kolor_glowny) {
  cm_df <- as.data.frame(cm)
  cm_df$Przewidywana <- factor(cm_df$Przewidywana, levels = rev(levels(cm_df$Przewidywana)))

  ggplot(cm_df, aes(x = Rzeczywista, y = Przewidywana, fill = Freq)) +
    geom_tile(color = "black", linewidth = 0.5) +
    geom_text(aes(label = Freq), size = 6, fontface = "bold") +
    scale_fill_gradient(low = "white", high = kolor_glowny) +
    theme_minimal() +
    coord_fixed() + # <-- DODANE: wymusza kwadratowe kafelki
    labs(title = tytul,
         x = "Rzeczywista",

```

```

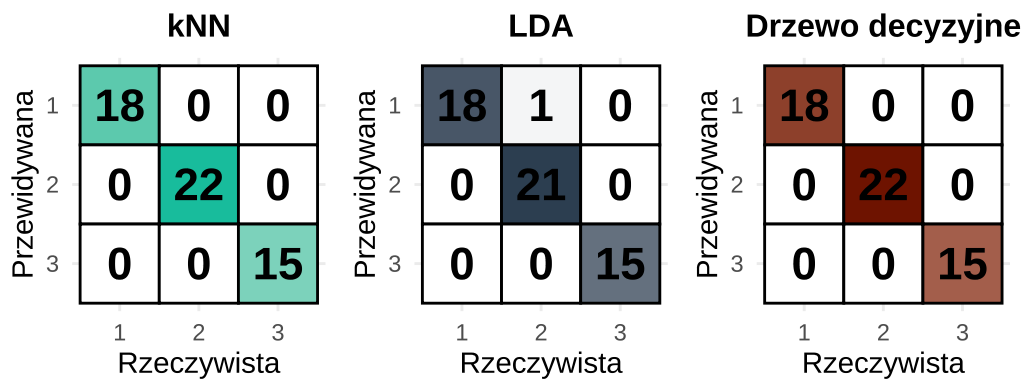
    y = "Przewidywana") +
    theme(plot.title = element_text(face = "bold", size = 12, hjust = 0.5),
          legend.position = "none")
}
wykres_cm_knn <- rysuj_macierz(cm_knn, "kNN", "#18BC9C")
wykres_cm_lda <- rysuj_macierz(cm_lda, "LDA", "#2C3E50")
wykres_cm_tree <- rysuj_macierz(cm_tree, "Drzewo decyzyjne", "#6E1300")

wykres_zbiorczy <- wykres_cm_knn + wykres_cm_lda + wykres_cm_tree +
  plot_layout(ncol = 3) +
  plot_annotation(
    title = "Porównanie Macierzy Pomyłek na zbiorze testowym",
    theme = theme(plot.title = element_text(size = 16, face = "bold", hjust = 0.5))
  )

print(wykres_zbiorczy)

```

Porównanie Macierzy Pomyłek na zbiorze testowym



```

df_plots_pca <- data.frame(Class = Y_train_full, pca_full$x[, 1:2])

ggplot(df_plots_pca, aes(x = PC1, y = PC2, color = Class)) +
  geom_point(size = 3, alpha = 0.8) +
  theme_minimal() +
  scale_color_manual(values = c("1" = "#2C3E50", "2" = "#18BC9C", "3" = "#E74C3C")) +
  labs(title = "Wizualizacja zbioru Wine w przestrzeni dwóch pierwszych składowych głównych",
       subtitle = "Wyraźna separacja geometryczna klas wyjaśnia wysoką skuteczność modeli kNN",
       x = "Pierwsza składowa główna (PC1)",
       y = "Druga składowa główna (PC2)")

```


Wizualizacja zbioru Wine w przestrzeni dwóch pierwszych składowych
Wyraźna separacja geometryczna klas wyjaśnia wysoką skuteczność modelu

